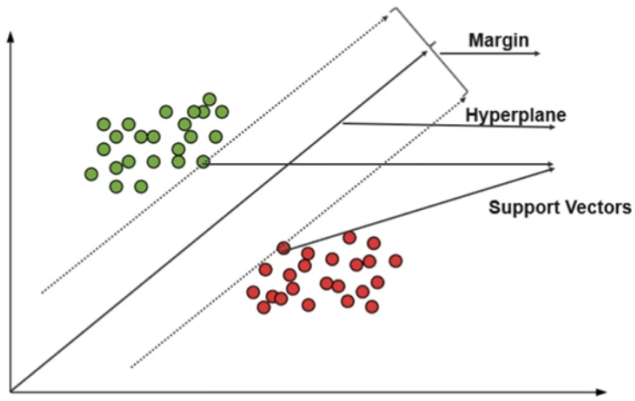
0.1 Support Vector Machine (SVM)

0.1.1 Bài toán phân loại tuyến tính

Giả sử chúng ta có hai lớp (class) khác nhau được mô tả bởi các điểm trong không gian nhiều chiều. Trong trường hợp hai lớp này là phân tách tuyến tính (*linearly separable*), tồn tại một siêu phẳng (hyperplane) có thể phân chia chính xác các điểm dữ liệu thuộc về hai lớp đó.



Hình 0.1: Minh họa siêu phẳng phân chia trong không gian hai chiều.

Trên thực tế, có thể tồn tại nhiều mặt phân chia khác nhau. Thuật toán SVM tìm kiếm một siêu phẳng tối ưu sao cho khoảng cách từ điểm gần nhất của mỗi lớp đến mặt phân chia là bằng nhau và khoảng cách đó (gọi là lề - *margin*) đạt giá trị lớn nhất có thể.

0.1.2 Xây dựng bài toán tối ưu

Giả sử tập dữ liệu huấn luyện bao gồm các cặp dữ liệu $(\mathbf{x}_n, y_n)$ với $n = 1, 2, \ldots, N$, trong đó $\mathbf{x}_n \in \mathbb{R}^d$ là vector đầu vào d chiều và $y_n \in \{1, -1\}$ là nhãn.

Mục tiêu của thuật toán SVM là tìm bộ tham số $(\mathbf{w}, b)$ sao cho khoảng cách từ các điểm gần siêu phẳng nhất đến siêu phẳng đó là lớn nhất. Giả thuyết về lề hàm số cho phép chuẩn hóa sao cho $y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1$ cho mọi điểm, dẫn đến bài toán tối ưu chuẩn tắc:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 \quad \text{thỏa mãn } 1 - y_n\left(\sum_{j=1}^{d} w^{(j)} x_n^{(j)} + b\right) \leq 0, \forall n \tag{1}$$

0.1.3 Bài toán đối ngẫu

Để giải bài toán tối ưu lồi với các ràng buộc bất đẳng thức, chúng ta sử dụng phương pháp nhân tử Lagrange để chuyển sang bài toán đối ngẫu. Điều kiện Slater được thỏa mãn, đảm bảo tính chất đối ngẫu mạnh.

Bài toán đối ngẫu cuối cùng là:

$$\boldsymbol{\lambda}^* = \arg\max_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}) \tag{2}$$

thỏa mãn:

- $\lambda_n \geq 0, \quad \forall n = 1, 2, \ldots, N$
- $\sum_{n=1}^{N} \lambda_n y_n = 0$

a, Vector hỗ trợ và Xác định tham số mô hình

Những điểm dữ liệu ứng với $\lambda_n > 0$ được gọi là các **Vector hỗ trợ (Support Vectors)**. Đây là những điểm nằm gần siêu phẳng phân chia nhất và trực tiếp quyết định vị trí của siêu phẳng đó.

Sau khi giải được $\boldsymbol{\lambda}$ bằng thư viện tối ưu, ta xác định các tham số của mô hình như sau:

1. **Tìm $\mathbf{w}$:** Thay các giá trị λ_n vừa tìm được vào công thức đạo hàm:

$$\mathbf{w}^* = \sum_{n \in \mathcal{S}} \lambda_n y_n \mathbf{x}_n \tag{3}$$

   với $\mathcal{S}$ là tập chỉ số của các Support Vectors.

2. **Tìm b:** Với bất kỳ một Support Vector $\mathbf{x}_k$ ($k \in \mathcal{S}$), ta có phương trình $y_k(\mathbf{w}^T\mathbf{x}_k + b) = 1$. Nhân cả hai vế với y_k (do $y_k^2 = 1$), ta thu được:

$$b^* = y_k - \mathbf{w}^{*T}\mathbf{x}_k = y_k - \sum_{j=1}^{d} w^{*(j)} x_k^{(j)} \tag{4}$$

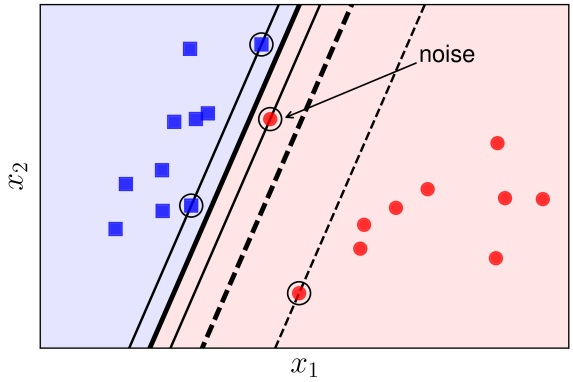
   Để kết quả ổn định hơn về mặt số học, giá trị b^* thường được tính bằng cách lấy trung bình cộng trên toàn bộ tập Support Vectors $\mathcal{S}$.

0.1.4 Soft Margin Support Vector Machine

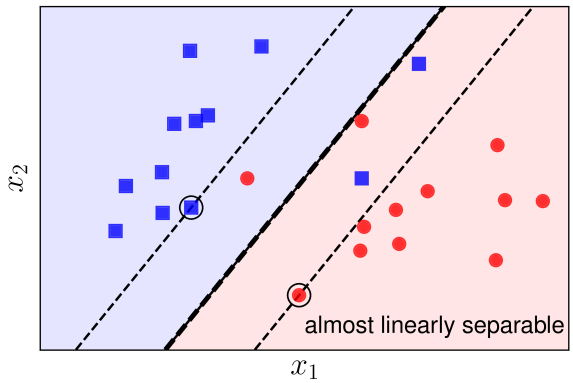
Trong thực tế, phương pháp Hard Margin SVM gặp phải hai hạn chế lớn: dữ liệu phân tách tuyến tính nhưng có nhiễu, hoặc dữ liệu không phân tách tuyến tính.

Để giải quyết, khái niệm Soft Margin được đưa ra. Với mỗi điểm dữ liệu $\mathbf{x}_n$, chúng ta giới thiệu biến bù $\xi_n \geq 0$ đại diện cho độ vi phạm. Ràng buộc mềm là $y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1 - \xi_n$.

Tham số $C > 0$ cân bằng giữa tối đa hóa lề và cực tiểu hóa sai số. Bài toán tối



Hình 0.2: Ảnh hưởng của điểm nhiễu đối với Hard Margin SVM.



Hình 0.3: Dữ liệu không thể phân tách tuyến tính.

ưu là:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{n=1}^{N} \xi_n \tag{5}$$

Bài toán đối ngẫu cuối cùng:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m x_n^T x_m \right) \tag{6}$$

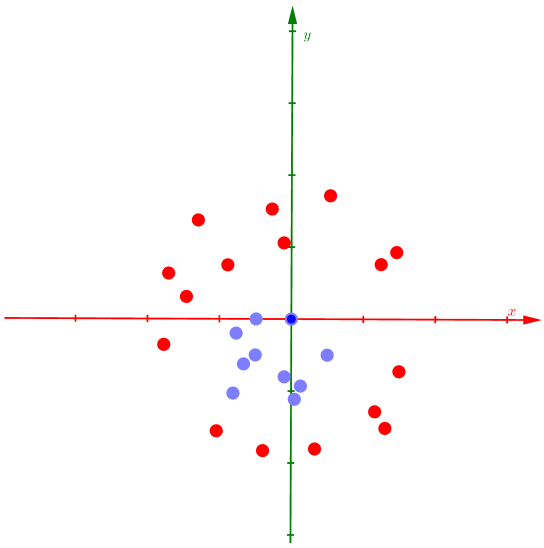
với $0 \leq \lambda_n \leq C$ và $\sum \lambda_n y_n = 0$.

Phân tích vị trí các điểm dữ liệu dựa trên KKT:

- $\lambda_n = 0$: Điểm nằm trong vùng an toàn (đúng phía và ngoài lề).
- $0 < \lambda_n < C$: Điểm nằm chính xác trên đường margin.
- $\lambda_n = C$: Điểm vi phạm margin (nhiễu hoặc phân lớp sai).

0.1.5 Kernel Support Vector Machine

Kernel SVM giải quyết vấn đề khi dữ liệu không phân tách tuyến tính bằng cách ánh xạ sang không gian cao chiều.



Hình 0.4: Dữ liệu phi tuyến tính trong không gian 2 chiều.

Ý tưởng là ánh xạ dữ liệu qua hàm $\Phi(\mathbf{x})$ để phân tách tuyến tính trong không gian mới.

Hàm Kernel $K(\mathbf{x}, \mathbf{z}) = \Phi(\mathbf{x})^T \Phi(\mathbf{z})$ cho phép tính tích vô hướng mà không cần ánh xạ trực tiếp.

Bài toán tối ưu với Kernel:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m K(\mathbf{x}_n, \mathbf{x}_m) \right) \tag{7}$$

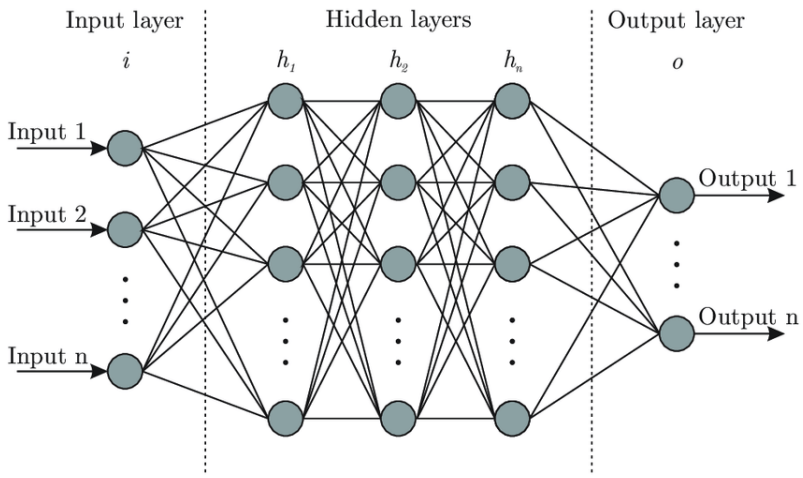
Công thức dự đoán nhãn:

$$y_{new} = \text{sgn} \left(\sum_{m \in S} \lambda_m y_m K(\mathbf{x}_m, \mathbf{x}) + b \right) \tag{8}$$

0.2 Convolutional Neural Network (CNN)

0.2.1 Mạng nơ-ron truyền thống (Fully Connected Network)

Trước khi đi sâu vào kiến trúc CNN, ta xem xét mô hình mạng nơ-ron truyền thống, nơi mỗi nơ-ron ở tầng này kết nối với toàn bộ nơ-ron ở tầng kế tiếp.



Hình 0.5: Mô hình mạng nơ-ron truyền thống với các tầng liên kết đầy đủ.

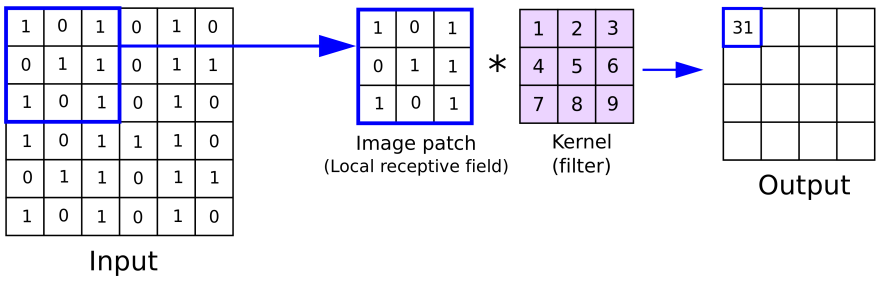
Tuy nhiên, đối với dữ liệu hình ảnh, mạng liên kết đầy đủ có hai hạn chế lớn: sự bùng nổ tham số do số lượng kết nối lớn, và mất thông tin không gian khi flatten ảnh thành vector.

0.2.2 Mạng nơ-ron tích chập (CNN)

CNN giải quyết các nhược điểm trên bằng cách sử dụng kết nối cục bộ và chia sẻ trọng số.

a, Kernel và Bias

CNN sử dụng bộ lọc (Kernel) trượt trên ảnh để thực hiện phép nhân chập.



Hình 0.6: Quá trình trượt Kernel trên ảnh đầu vào để trích xuất đặc trưng.

Công thức tích chập:

$$O_{i,j} = \sum_m \sum_n (I_{i+m,j+n} \cdot K_{m,n}) + b \tag{9}$$

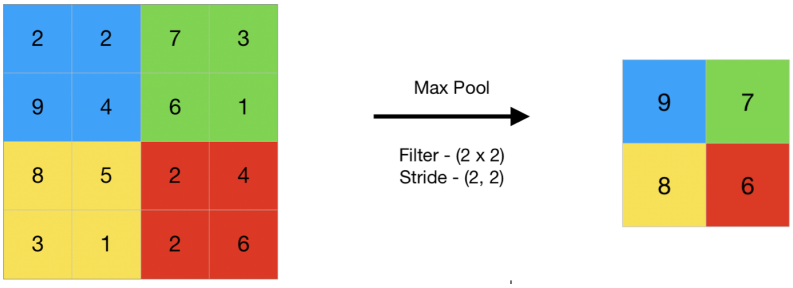
b, Hàm kích hoạt (Activation Function)

Hàm ReLU được sử dụng phổ biến:

$$f(x) = \max(0, x) \tag{10}$$

c, Lớp lấy mẫu cực đại (Max Pooling)

Max Pooling giảm kích thước dữ liệu bằng cách lấy giá trị lớn nhất trong vùng.



Hình 0.7: Minh họa cơ chế Max Pooling giúp giảm chiều dữ liệu.

d, Hàm mục tiêu và Phân loại (Loss Function)

Hàm Softmax chuyển logits thành xác suất:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \tag{11}$$

Hàm mất mát Cross-Entropy:

$$L = -\sum_{i=1}^{n} y_i \log(\hat{y}_i) \tag{12}$$

Mục tiêu là tối thiểu hóa L.

e, Lan truyền ngược (Backpropagation)

Quá trình lan truyền ngược sử dụng quy tắc chuỗi để tính đạo hàm và cập nhật tham số.

Công thức cập nhật:

$$K_{new} = K_{old} - \eta \cdot \frac{\partial L}{\partial K} \tag{13}$$

$$b_{new} = b_{old} - \eta \cdot \frac{\partial L}{\partial b} \tag{14}$$